

AI HR Operations Employee

Product Scope • AI Agent Architecture • Technology Stack • Free Deployment • Roadmap

Portfolio project: an agentic AI employee for HR back-office operations. The system converts natural-language HR requests into validated, auditable workflows using policy RAG, typed tools, verification, and human approval.

Dimension	Target
Product	AI-powered HR back-office operations platform
Primary user	HR administrator / HR operations executive
AI role	Digital HR Operations Employee
Core interaction	Natural language → plan → tools → verification → audit
Deployment goal	Public portfolio application
Target cost	₹0/month within free-tier limits
Repository	Single monorepo

Core principle: the AI does not merely answer questions. It understands intent, plans work, retrieves applicable policy, calls approved tools, verifies the resulting state, and records what happened.

1. Product Vision

Build a realistic HR administration application where repetitive back-office work can be delegated to an AI employee while humans retain control over sensitive decisions.

Example

“Onboard Priya as a Software Engineer joining October 12, reporting to Rahul in Bangalore.”

- Identify or create the employee.
- Validate department, manager, location, and joining date.
- Retrieve the onboarding policy.
- Determine required documents and tasks.
- Create the onboarding workflow.
- Ask for missing information.
- Request human approval for sensitive actions.
- Execute through controlled tools.
- Verify the final state.
- Write an audit trail.

2. Product Goals

- Create a genuine software product rather than a chatbot demo.
- Demonstrate agentic AI: planning, tool calling, state management, verification, recovery, and human-in-the-loop.
- Use RAG for HR policies and knowledge rather than for deterministic business rules.
- Keep permissions and business rules outside the LLM.
- Make AI actions observable, measurable, and auditable.
- Deploy a usable public demo with free-tier infrastructure.

3. HR Operations Scope

3.1 Employee Management

- Employee profile: identity, contact, employment, department, job title, manager, location, joining date, status.
- Create, edit, search, archive, reactivate, and view employees.
- Employee document inventory and verification status.
- Role and permission assignment.
- Bulk operations with preview and approval.

AI examples: “Create Priya as a Software Engineer.” / “Move Arun to Product.” / “Show employees whose probation ends next month.” / “Find incomplete employee profiles.”

3.2 Employee Onboarding

- Create employee record.
- Assign department, manager, role, and location.
- Create onboarding checklist.
- Track required documents.
- Detect missing information.

- Generate/prepare onboarding documents.
- Prepare welcome communication.
- Track onboarding progress and completion.

3.3 Leave Management

- Create and view leave requests.
- View balances.
- Approve/reject according to permissions.
- Validate dates, conflicts, and applicable policy.
- Generate leave summaries.
- Explain why a request cannot be approved.

3.4 Attendance Operations

- Daily/monthly attendance views.
- Missing attendance detection.
- Attendance anomaly detection.
- Cross-check attendance with approved leave.
- Prepare correction proposals.
- Require approval before changing historical records.

3.5 HR Documents

- Upload employee documents.
- OCR and structured extraction.
- Document classification.
- Cross-document comparison.
- Missing-field and identity mismatch detection.
- Verification workflow and audit history.

Example: if a name differs between an offer letter, bank document, and tax document, flag it for review instead of automatically correcting it.

3.6 Payroll Preparation — Initial Scope

- Do not build full payroll calculation in the MVP.
- Prepare payroll inputs from employee changes, attendance, leave, and compensation records.
- Detect missing bank details and inconsistent employee data.
- Generate a reviewable preparation report.

3.7 AI HR Command Center

- Natural-language request input.
- Intent summary and extracted entities.
- Execution plan preview.
- Missing-information prompts.
- Policy evidence.
- Risk classification.
- Approval cards.

- Live tool execution timeline.
- Verification result.
- Audit/trace view.

4. AI Employee Architecture

Component	Responsibility
Intent / Router	Understand the request and identify the HR operation.
Planner	Break the request into ordered actions.
Policy RAG	Retrieve applicable HR policy and evidence.
Tool Selector	Choose approved tools.
Executor	Call secure application APIs.
Verifier	Confirm expected business state after execution.
Recovery	Handle failures, retries, partial completion, and safe compensation.
Human Approval	Pause sensitive or irreversible operations.
Audit / Trace	Record request, plan, tools, results, approvals, and final state.

5. Agent Execution Flow

Request: “Onboard Priya as a Software Engineer joining October 12, reporting to Rahul.”

- Understand: extract intent and entities; resolve whether Priya already exists.
- Retrieve: retrieve onboarding policy and required-document rules.
- Plan: create employee → set employment details → assign manager/department → create checklist → request missing documents → prepare welcome message.
- Validate: permissions, dates, manager, department, required fields.
- Approve: pause on configured medium/high-risk actions.
- Execute: call strongly typed tools through the HR API.
- Verify: re-read the affected state and confirm each expected result.
- Audit: persist the complete execution trace.

6. AI Tool Catalog

Tool	Purpose	Approval
search_employee	Find employee by ID, name, email, or attributes.	No
create_employee	Create employee profile.	Configurable
update_employee	Update non-sensitive fields.	Usually no
change_manager	Change reporting manager.	Recommended
change_department	Transfer employee/team.	Recommended
create_leave_request	Create leave request.	Configurable
approve_leave	Approve leave.	Yes in approval workflow
read_attendance	Read attendance data.	No
propose_attendance_correction	Prepare correction without applying it.	No

Tool	Purpose	Approval
apply_attendance_correction	Apply approved correction.	Yes
search_policy	Retrieve policy evidence.	No
upload_document	Store document/file metadata.	Configurable
extract_document	Run OCR/document extraction.	No
compare_documents	Compare extracted fields.	No
generate_document	Prepare HR document.	Configurable
send_email	Send approved HR communication.	Yes initially

7. HR Policy RAG

RAG is used for policies and knowledge, not for permissions or deterministic business rules.

- Sources: PDF, DOCX, Markdown, and text policy documents.
- Pipeline: parse → chunk → metadata → embeddings → pgvector → retrieval → optional reranking → LLM.
- Metadata: policy category, effective date, version, department, jurisdiction.
- Show supporting evidence in the UI when a policy affects a workflow.
- Keep policy versions so historical decisions remain traceable.

8. Technology Stack

Layer	Technology	Purpose
Frontend	Next.js + TypeScript	HR admin UI + AI command center
UI	Tailwind CSS + shadcn/ui	Enterprise-style components
Data fetching	TanStack Query	API state/caching
Backend	NestJS + TypeScript	HR business logic and tools
Database	PostgreSQL	Transactional HR data
Vector search	pgvector	Policy/document semantic search
ORM	Prisma	Type-safe database access
AI service	Python + FastAPI	AI runtime
Agent orchestration	LangGraph	Stateful multi-step agent workflows
LLM layer	OpenAI-compatible abstraction + free/local provider	Model flexibility
Embeddings	Sentence Transformers / local embedding model	RAG embeddings
OCR	PaddleOCR or Tesseract	Document extraction
Redis/queue	Upstash Redis + BullMQ	Background jobs/caching
File storage	Supabase Storage	HR documents
Auth	Auth.js / NextAuth	Authentication
Authorization	Backend RBAC	Permissions
AI observability	Langfuse	Traces, latency, generations, evaluation
App observability	OpenTelemetry	Distributed traces/metrics
Testing	Vitest + Playwright + Python tests	Unit/API/E2E/AI tests

Layer	Technology	Purpose
Containers	Docker + Docker Compose	Reproducible local environment
Deployment	Cloudflare Pages + Render + Supabase + Upstash	Free-tier public deployment
CI	GitHub Actions	Build/test/deploy automation

9. Free Deployment Architecture

Service	Use	Notes
Cloudflare Pages	Next.js frontend	Public web UI; free-tier limits apply.
Render	NestJS API + FastAPI AI service	Free services may sleep when idle and have limited compute.
Supabase	PostgreSQL + pgvector + storage	Demo-scale HR data; free quotas apply.
Upstash	Redis	Use only where needed; stay within free request limits.
Langfuse Cloud	AI observability	Free tier suitable for portfolio-scale usage.
Hosted/free LLM inference	Public AI runtime	Keep provider replaceable; free quotas can be rate-limited.

Local fallback: Ollama can run a supported open model during development. The public deployment should use a hosted/free inference option rather than trying to run a large local model on free web-service compute.

10. Core Data Model

Entity	Important fields
Employee	id, employeeCode, name, email, DOB, departmentId, managerId, jobTitle, location, joiningDate, status
Department	id, name, managerId, status
LeaveRequest	id, employeeId, type, startDate, endDate, reason, status, approverId
AttendanceRecord	id, employeeId, date, checkIn, checkOut, status, correctionReason
Document	id, employeeId, type, storageKey, status, uploadedAt, verifiedAt
PolicyDocument	id, title, version, effectiveFrom, category, storageKey
PolicyChunk	id, policyId, content, embedding, metadata
OnboardingTask	id, employeeId, taskType, status, assignee, dueDate
WorkflowRun	id, requestId, workflowType, status, startedAt, completedAt
AgentRun	id, workflowRunId, model, promptVersion, status, latency
ToolCall	id, agentRunId, toolName, input, output, status, timestamp
Approval	id, toolCallId, approverId, status, reason, timestamp
AuditLog	id, actorType, actorId, action, entityType, entityId, before, after, timestamp

11. Security & Safety

- LLM has no unrestricted database credentials.
- Every state-changing operation is an explicit, typed tool.
- Backend RBAC is enforced independently of the model.
- Tool inputs are schema-validated before execution.

- Sensitive fields are masked in traces where possible.
- High-impact operations require human approval.
- Every mutation creates an audit event.
- Use idempotency keys for retryable actions.
- Use version checks/optimistic locking for concurrent updates.
- Never invent missing employee data.
- Document conflicts go to manual review.
- Keep proposed actions separate from executed actions.

Risk	Examples	Default behavior
Low	Read employee, search policy, create checklist	May execute automatically
Medium	Change manager, department transfer, send email	Configurable approval
High	Salary, bank details, termination, payroll changes	Mandatory human approval

12. AI Evaluation

Create a synthetic evaluation set of roughly 50–100 HR requests covering normal cases, ambiguity, missing information, conflicting documents, permission violations, and tool failures.

- Intent accuracy.
- Entity extraction accuracy.
- Tool-selection accuracy.
- Tool-argument accuracy.
- Policy retrieval recall.
- Groundedness of policy-dependent responses.
- Workflow completion rate.
- Verification accuracy.
- Approval correctness.
- Regression performance across prompt/model changes.

13. Testing Strategy

Layer	Examples
Unit	Leave validation, permission checks, state transitions
API	Employee, leave, attendance, document endpoints
Agent	Intent → plan → correct tool selection
RAG	Policy retrieval and evidence correctness
Tool safety	Unauthorized tool calls fail
Workflow	Onboarding completes across multiple steps
Failure	Timeouts, duplicate requests, partial execution
E2E	Natural-language request → verified HR state

14. Repository Structure

```
hr-ai-employee/ ├── apps/ | ├── web/ # Next.js HR admin UI | ├── api/ # NestJS HR backend | └── ai/ # FastAPI + LangGraph agent
├── packages/ | ├── db/ # Prisma schema/client | ├── contracts/ # Shared DTOs/schemas | ├── ui/ # Shared UI | └── config/ #
Shared configuration ├── ai/ | ├── agents/ ├── graphs/ ├── tools/ | ├── prompts/ ├── rag/ ├── evaluators/ | └── guardrails/ ├── docs/
```

15. Development Roadmap

Phase	Scope	Outcome
1. Foundation	Monorepo, auth, DB, employee CRUD, RBAC	Working HR admin
2. Core HR	Leave, attendance, onboarding, documents	Usable HR operations app
3. AI Command Center	LLM gateway, intent parsing, structured requests	Natural-language interface
4. Tool Calling	Typed tools + permission checks	Agent performs real actions
5. LangGraph	Planning, execution, verification, recovery	True agentic workflow
6. RAG	Policy ingestion + pgvector + evidence UI	Grounded workflows
7. Document AI	OCR + extraction + conflict detection	Automated document operations
8. Human-in-loop	Risk classes + approvals	Safe execution
9. Evaluation	Dataset + regression + traces	Measurable AI quality
10. Deployment	Cloudflare + Render + Supabase + Upstash + Langfuse	Public portfolio demo

16. MVP Boundary

Build first: employee management, departments/managers, leave, onboarding checklist, document upload/extraction, AI command center, 5–8 safe tools, LangGraph planner/executor/verifier, policy RAG, approval flow, audit trail, and public deployment.

Defer: full payroll, banking integrations, statutory compliance, multi-company tenancy, complex workflow builder, enterprise SSO, and autonomous high-risk operations.

17. Interview / Portfolio Demo

Demo scenario	What it demonstrates
“Onboard Priya”	Intent, planning, RAG, tools, multi-step execution
Conflicting documents	OCR, extraction, cross-document reasoning, safe review
Leave approval	Policy retrieval + deterministic validation + approval
Sensitive change	Risk classification + RBAC + human-in-loop
Tool failure	Recovery + retry + verification + auditability
Evaluation dashboard	AI quality measurement and regression testing

18. Portfolio Positioning

Suggested description: “An agentic AI HR Operations Employee that converts natural-language HR requests into validated, auditable business workflows. It combines LangGraph-based planning, policy RAG, typed tool calling, document intelligence, human approval, state verification, and AI observability.”

Skills demonstrated: full-stack engineering, agentic AI, RAG, tool calling, workflow orchestration, document AI, security, evaluation, observability, and public deployment.

19. Key Architecture Decisions

Decision	Reason
Separate AI service from HR API	AI experimentation stays independent from deterministic business logic.

Decision	Reason
LangGraph for workflows	Stateful multi-step behavior is explicit and testable.
PostgreSQL + pgvector	Avoids unnecessary vector infrastructure.
LLM provider abstraction	Supports free/local models now and hosted models later.
AI tools call application APIs	Prevents unrestricted LLM database access.
Verification after actions	Tool success does not guarantee correct business state.
Human approval for sensitive actions	Controls risk and demonstrates responsible agent design.
Single monorepo	Simple cloning, development, CI, and deployment.

20. Definition of Done

- HR users can manage core records through the normal admin UI.
- Selected HR operations can be performed through natural language.
- The agent produces an explicit plan for multi-step tasks.
- Policy-dependent workflows retrieve supporting evidence.
- Every AI mutation uses a typed backend tool.
- Sensitive operations require human approval.
- The agent verifies the final state.
- Failures/retries are visible.
- Agent runs are auditable.
- An evaluation suite measures quality.
- The project runs locally with Docker Compose.
- A public demo is deployed within free-tier limits.